

Le Mot Juste

Jeu *Motus* en architecture microservices

Master 2 MIAGE SITN — Applications Web orientées Services

Enseignant : M. Mouloud Menceur

Binôme : Omar Soliman • Abderrahmane Tsouli

Dépôt GitHub : <https://github.com/omarsoliman02/LeMotJuste>

Juillet 2026

1 Présentation

"Le Mot Juste" est une implémentation du jeu **Motus** sous forme de microservices Spring Boot. Le joueur démarre une partie, reçoit un mot mystère (longueur connue, première lettre révélée) et dispose de **6 essais** pour le deviner. À chaque proposition, le jeu indique, lettre par lettre, ce qui est bien placé, mal placé ou absent. Les résultats sont historisés et donnent lieu à des statistiques et à un classement.

L'application est découpée par **domaine métier** : gestion des joueurs, logique du jeu, et historisation des scores. Un point d'entrée unique (la *gateway*) expose l'ensemble au client, qui ne communique jamais directement avec les services. Une page web de démonstration, dont l'interface s'inspire de *Wordle*, permet de jouer entièrement depuis le navigateur.

2 Architecture

Le système repose sur quatre services Spring Boot et une instance PostgreSQL. La communication interne se fait en **OpenFeign synchrone**. Chaque service est **propriétaire de sa base** : aucun accès croisé direct, toute interaction passe par l'API REST ou Feign.

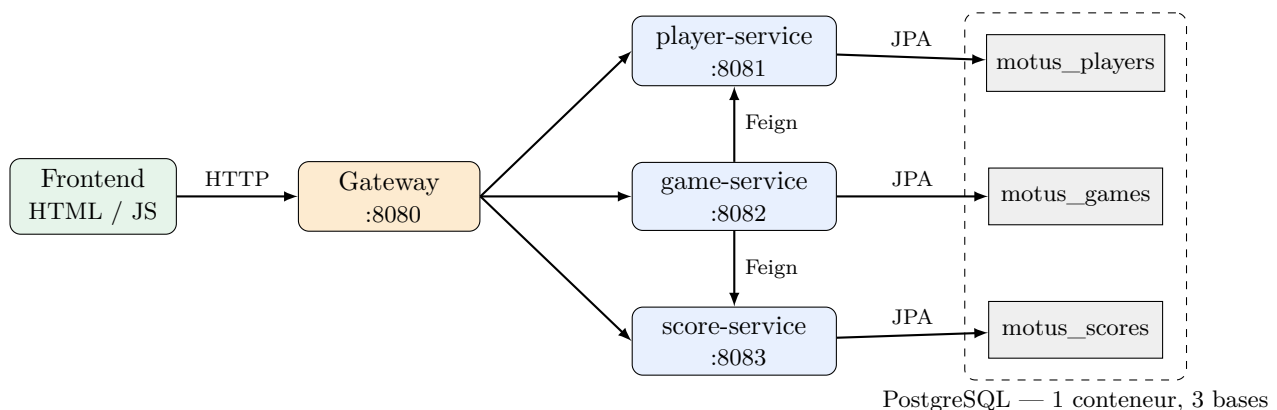


FIGURE 1 – Schéma d'architecture. La gateway route `/api/players`, `/api/games` et `/api/scores`. `game-service` appelle `player-service` ("le joueur existe-t-il?") et `score-service` ("enregistrer le résultat").

Rôle des services.

- **gateway (8080)** — Spring Cloud Gateway, point d'entrée unique, routage et CORS.
- **player-service (8081)** — enregistrement et gestion des joueurs (base `motus_players`).
- **game-service (8082)** — logique Motus : mot mystère, validation, calcul lettre par lettre, dictionnaire (base `motus_games`).

— **score-service (8083)** — historique des parties, statistiques et classement (base `motus_scores`).

3 Diagramme de classes "métier"

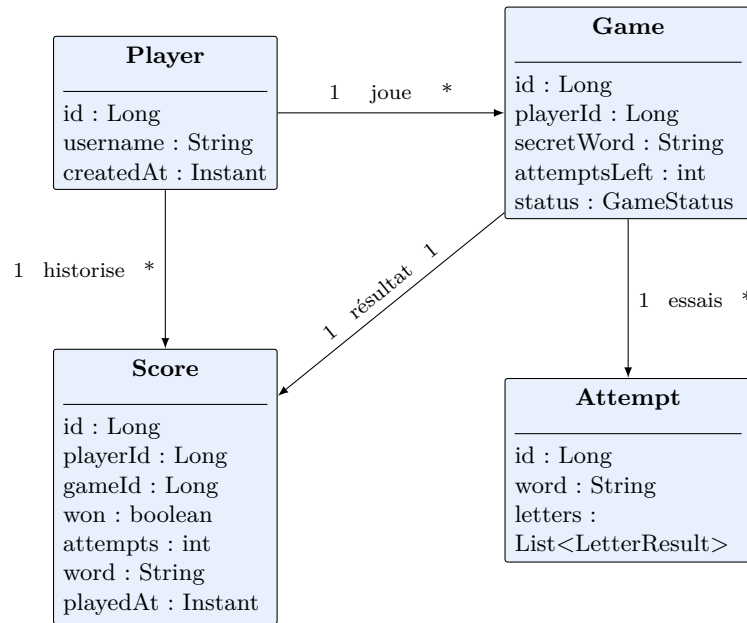


FIGURE 2 – Entités métier et relations. `Game.secretWord` n'est jamais renvoyé au client tant que la partie est `IN_PROGRESS`. L'historique des essais (`Attempt`) est calculé à la volée et conservé côté client ; seul le résultat final est persisté via `Score`.

4 Choix techniques

Élément	Choix
Langage	Java 21 (LTS)
Framework	Spring Boot 4.0.7
Spring Cloud (BOM)	2025.1.2 (Gateway + OpenFeign)
Build	Maven — un projet par service
Persistence	Spring Data JPA + PostgreSQL
Conteneurisation	Dockerfile multi-stage + docker-compose
Orchestration	Kubernetes / MiniKube
Frontend	HTML + JS vanilla, style Wordle

TABLE 1 – Stack technique.

Justification des principaux choix :

- **Découpage par domaine métier** (joueurs / jeu / scores) — couplage faible, déploiement indépendant, chaque service propriétaire de sa base.
- **Une seule instance PostgreSQL, trois bases logiques** — suffisant pour le périmètre du projet, évite la lourdeur de trois conteneurs.
- **OpenFeign synchrone** — simple et lisible, adapté aux flux "démarrer une partie" et "enregistrer un score".
- **Spring Cloud Gateway** — point d'entrée unique : centralise routage et CORS.
- **Frontend découplé** — page statique (HTML/JS) qui n'appelle que la gateway, sans dépendance ni outil de build ; interface inspirée de Wordle.

— **Périmètre volontairement réduit** — pas d'Eureka, de Config Server ni de broker de messages : on privilégie la lisibilité.

5 API REST

Toutes les routes sont exposées via la gateway (port 8080). Réponses et erreurs au format JSON homogène (timestamp, status, error, message).

Méthode	Chemin	Rôle
POST	/api/players	Créer un joueur (201 ; 409 si pris)
GET	/api/players/{id}	Lire un joueur
POST	/api/games	Démarrer une partie (vérifie le joueur)
POST	/api/games/{id}/guess	Proposer un mot (calcul des lettres)
GET	/api/games/{id}	État d'une partie (sans le mot mystère)
POST	/api/scores	Historiser un résultat (appelé en Feign)
GET	/api/scores?playerId=	Historique d'un joueur
GET	/api/scores/leaderboard	Classement

TABLE 2 – Principaux points d'entrée de l'API.

6 Algorithme Motus

Le cœur du jeu est le calcul lettre par lettre d'une proposition. Pour gérer correctement les **lettres en double**, il se fait en **deux passes** :

1. **Passé 1** — pour chaque position, si la lettre proposée est identique à celle du mot secret au même indice, elle est **CORRECT**. Les lettres non encore appariées du mot secret sont comptées dans une table de fréquences.
2. **Passé 2** — pour chaque position non **CORRECT**, si la lettre reste disponible dans la table, elle est **PRESENT** (et on décrémente le compteur), sinon **ABSENT**.

Exemple — mot secret **ALLER**, proposition **LELLE** : les deux L de la proposition sont bornés par le nombre réel de L restants après la passe 1, ce qui évite de signaler plus de lettres présentes qu'il n'en existe réellement. Toutes les chaînes sont normalisées (majuscules, sans accents) : le jeu est insensible à la casse et aux accents.

Validation et dictionnaire. Avant tout calcul, une proposition est rejetée (message en français, sans décompter d'essai) si sa longueur diffère de celle du mot mystère ou si le mot est inconnu. À la manière de Wordle, game-service s'appuie sur **deux listes** : une liste restreinte de mots courants dans laquelle est tirée la solution, et une liste beaucoup plus large de mots acceptés pour valider les propositions. Une proposition est valable si elle figure dans l'une des deux, ce qui permet de reconnaître bien plus de mots français tout en gardant des solutions usuelles.

7 Frontend (page de démo)

La page de démonstration est une application statique (HTML et JavaScript, sans framework ni build) dont l'interface s'inspire de *Wordle*. Elle ne communique qu'avec la gateway.

La page d'accueil (avant de choisir son pseudo) a été enrichie d'éléments visuels non textuels : une rangée de tuiles colorées façon Wordle en guise de bandeau, et trois pictogrammes illustrant en un coup d'œil les règles (indice de départ, six essais, code couleur). Un bouton **Règles** dans l'en-tête ouvre à tout moment une fenêtre récapitulant les règles du jeu, sans quitter l'écran en cours.

Le joueur saisit ou choisit un nom, puis **choisit la taille de la grille** (4 à 10 lettres, ou "Auto" pour un tirage aléatoire comme avant) avant de démarrer une partie : ce choix est transmis à `POST /api/games` via le champ optionnel `wordLength`, validé côté game-service par rapport aux bornes du dictionnaire. La longueur du mot et sa première lettre sont affichées, cette dernière apparaissant en gris clair sur la ligne en cours pour servir d'indice. À chaque proposition, la grille se colore lettre par lettre (vert : bien placée ; jaune : mal placée ; gris : absente) et un clavier AZERTY reflète l'état connu de chaque lettre. En fin de partie, le mot est révélé ; une fenêtre de statistiques présente l'historique du joueur et le classement (`/api/scores`). Les essais ne sont pas persistés côté serveur : la page conserve les réponses reçues pour redessiner la grille.

Une **vue admin**, accessible depuis l'en-tête, agrège en lecture seule les données des trois services (`GET /api/players`, l'endpoint `GET /api/games` qui liste toutes les parties tous joueurs confondus avec leur date de création, et `GET /api/scores`) : cartes de synthèse (nombre de joueurs, de parties, de parties en cours, taux de victoire global), classement des joueurs, et tableaux détaillés des parties et des scores, **filtrables par nom de joueur et par période** (recherche côté client, sans appel réseau supplémentaire puisque les données sont déjà chargées). L'accès est protégé par un mot de passe saisi sur un écran de connexion dédié ; comme Spring Security est hors périmètre du projet (cf. §8), cette protection reste côté client et ne remplace pas une authentification serveur.

8 Compilation et exécution

La procédure complète figure dans le `README.md` du dépôt. En résumé, tout se lance via Docker Compose (PostgreSQL et ses trois bases, les services et la gateway) :

```
docker compose up --build      # démarre toute la stack
# créer un joueur puis jouer :
curl -X POST localhost:8080/api/players -H 'Content-Type: application/json' \
  -d '{"username":"alice"}'
curl -X POST localhost:8080/api/games -H 'Content-Type: application/json' \
  -d '{"playerId":1}'
curl -X POST localhost:8080/api/games/1/guess -H 'Content-Type: application/json' \
  -d '{"word":"cheval"}'
```

La page de démo se lance ensuite avec `./serve.sh`, puis en ouvrant <http://localhost:5500>. Un déploiement sur **MiniKube** est également disponible (`kubectl apply -k k8s/` : un **Deployment** et un **Service** par composant, **Secret** pour les identifiants PostgreSQL, **ConfigMap** pour les URLs inter-services, gateway exposée en **NodePort**). Le guide détaillé, avec schémas d'architecture et script de démo, est dans `docs/kubernetes-guide.md`.

9 Bilan

Réussites. Un découpage microservices clair et homogène (un même patron repris pour chaque service), un contrat d'API figé qui a permis aux deux membres du binôme de travailler en parallèle sans se bloquer, un algorithme Motus robuste aux lettres en double (validé par des tests unitaires), et une interface web soignée inspirée de Wordle.

Difficultés. La gestion des doublons dans le calcul des lettres a demandé de la rigueur (d'où l'approche en deux passes). La configuration des versions récentes (Spring Boot 4, Spring Cloud Gateway réactif) et la communication inter-services (gestion des pannes : enregistrement du score en "best-effort") ont représenté l'essentiel de l'effort de mise au point. Côté jeu, le dictionnaire initial, trop restreint, refusait des mots français corrects : nous l'avons remplacé par deux listes (mots courants pour la solution, large liste pour la validation).

Ce que nous avons appris. L'architecture microservices et ses compromis, Spring Cloud Gateway, OpenFeign, la conteneurisation multi-stage avec Docker, la persistance JPA multi-bases et le déploiement sur MiniKube.

Ce que nous avons le plus / le moins aimé. Nous avons apprécié la modularité et l'autonomie de chaque service ; la contrepartie a été la lourdeur de configuration et de débogage propre aux échanges inter-services.